

Service Mesh — Easy KCNA Notes

Since you're learning this as a **beginner**, first remember:

Service Mesh = a dedicated system that manages communication between services.

It is especially useful when you have many **microservices**.

1. Why do we need a Service Mesh?

Imagine an application with many microservices:

User Service



Order Service



Payment Service



Product Service

These services need to communicate with each other.

As the number of services increases, networking becomes difficult.

You may need:

-  Security
-  Traffic management
-  Monitoring
-  Retries
-  Health checking
-  Load balancing

A **Service Mesh** helps manage these communication-related tasks.

2. Simple Real-Life Example 🚦

Imagine a city with thousands of cars.

You need:

🚦 Traffic lights

👮 Traffic police

🛣️ Roads

📊 Traffic monitoring

Similarly, in a microservices application:

Microservices



Service Mesh



Manages communication

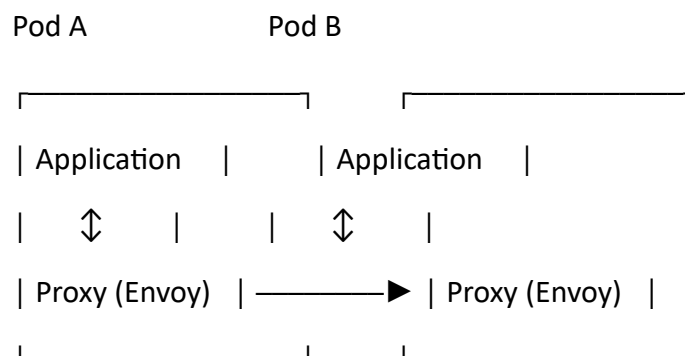
Easy memory:

Service Mesh = Traffic management system for microservices.

3. How does Service Mesh work?

A common service-mesh design uses a **proxy** alongside each application.

For example:



The application communicates through the proxy.

The proxy handles many networking concerns.

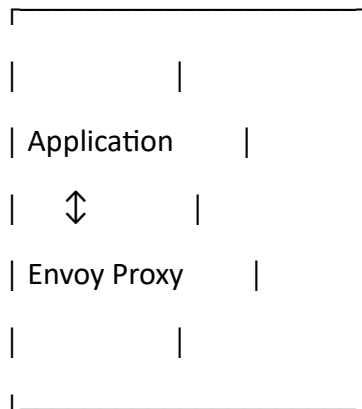
4. Envoy and Service Mesh ★

You just learned about **Envoy**.

This connects directly to Service Mesh.

Envoy can act as the **proxy** for a service.

Pod



So remember:

Envoy = Proxy

Service Mesh = System for managing service-to-service communication

Envoy is one technology commonly used to implement the proxy/data-plane part of a service mesh.

5. What does a Service Mesh provide?

Security

Can help secure communication between services.

Example:

Service A  —————  Service B

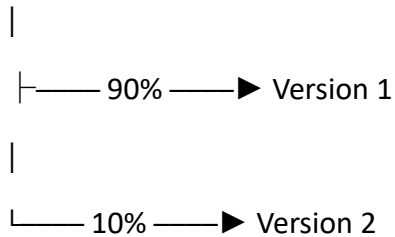
It can support encrypted service-to-service communication.

Traffic Management

It can control how traffic moves between services.

Example:

Service A



This can be useful for gradual releases.

Retries

If a request fails, the proxy can potentially retry it according to configured policies.

Request



Service B ❌



Retry



Service B ✅

Timeouts

A service mesh can configure how long a request should wait.

Request



Wait



Timeout 🕒

This prevents requests from waiting forever.

Observability

It can provide information about service communication:

- Requests
- Errors
- Latency
- Traffic

Example:

Service A → Service B

1,000 requests

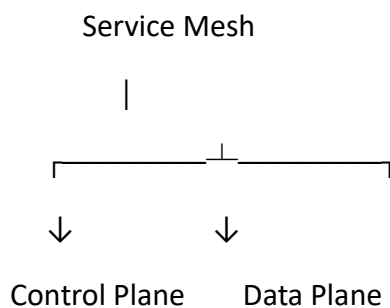
20 errors

100 ms latency

6. Control Plane and Data Plane ★★

This is an important concept.

A service mesh is commonly divided into **two parts**:



Control Plane

The **control plane** manages and configures the proxies.

Think:

Control Plane = Brain 🧠

It tells proxies what rules/policies to follow.

🚦 **Data Plane**

The **data plane** handles the actual application traffic.

It consists of the proxies.

Think:

Data Plane = Traffic 🚦

Control Plane

|

| configuration



Envoy Proxies

|

| actual traffic



Microservices

7. Service Mesh vs Kubernetes ★

Don't confuse them.

Kubernetes

Manages the **containers/workloads and cluster**.

Kubernetes



Pods

Deployments

Services

Nodes

Service Mesh

Focuses mainly on **service-to-service communication**.

Service Mesh



Traffic

Security

Observability

Retries

Timeouts

They can work **together**.

8. Service Mesh vs Service

These are also different.

Kubernetes Service

Provides a stable way to reach a set of Pods.

Client



Service



Pods

Service Mesh

Manages communication **between services** and provides additional networking features.

Service A



Service Mesh



Service B

KCNA Quick Revision

Service Mesh

-  Manages **service-to-service communication**
- Commonly used with **microservices**
- Helps with:
 -  Security
 -  Traffic management
 -  Observability
 -  Retries
 -  Timeouts
- Often uses **sidecar proxies**
- **Envoy** is a popular proxy used in service meshes.
- **Control Plane** → manages/configures proxies
- **Data Plane** → handles actual network traffic

★ One-line definition

A Service Mesh is a dedicated infrastructure layer that manages communication between microservices.

Easy memory

Kubernetes → manages your workloads

Service Mesh → manages communication between those workloads

Envoy → acts as the traffic proxy